

Change Report

Cohort 3 Team 1: Pixels of Promise

Movitz Aaron, Jake Adams, Oliver Belam, Anna Burt, Emily Foran, Sky
Haines-Bass, Job Young

University of York

ENG1: Software & Systems Engineering

January 13, 2025

1 Introduction

a.) Briefly summarise the processes, tools and conventions your team used to plan, make, keep track of, and review changes to the Assessment 1 deliverables & documentation.

1.1 Approach to Updating Assessment 1 Deliverables

To manage the Assessment 1 deliverables and documentation, our team followed structured approaches, each approach dependent on the deliverable being changed. Regardless of the deliverable being updated, the original team members that worked on our versions of those documents conducted initial brainstorming sessions. The purpose of these sessions was to outline where the past team succeeded and areas that required improvement. To either improve readability, maintainability, and overall reflected Assessment 2. After this at least two people from each original deliverable were assigned to be responsible for executing the changes and considerations mentioned as a whole, this ended up being four people total because each set of two had worked on two deliverables. One pair was assigned to work through *“Method Selection & Planning”* and *“Requirements”* whilst the other pair focused on *“Risk Assessment & Mitigation”* and *“Architecture”*.

1.2 Tracking & Managing Changes to Assessment 1 Deliverables

Progress for what sections of deliverables were covered was tracked on the overall Jira project board, with updates given during meetings. Minor changes were discussed in the *Change Report* group chat, while major updates, mainly when one of the four deliverable changes were complete were shared in the general group chat. Within those sections, each pair, especially with *“Risk Assessment & Mitigation”* and *“Requirements”*, duplicated the original tables, adding them underneath to monitor the changes better in the document. For the *“Requirements”* table, the pair highlighted requirements needing improvement in red and discussed potential changes during in-person meetings. When working outside of meetings, members used suggestion mode and document comments to propose changes with reasonings, waiting for feedback from other members. A requirement was unhighlighted when other team members agreed to the changes. The process of peer review with feedback via comments on the document was our main source of managing change within each deliverable. This ensured accuracy and consistent quality throughout the documents.

1.3 Version Control Practices

To ensure that changes could be tracked and referenced in Assessment 2 deliverables, such as Testing, the team implemented clear document naming conventions. This included referencing numbers for the requirements and risk tables, and the use of descriptive subtitles such as *“Assessment 2 [insert the original section header]”* where Assessment 2 content was applied. Version control was maintained throughout the process with these consistent name practices ensuring traceability to the original work of *Team 3: Time Complexity* in each deliverable. For example in the requirements document, a legend was used to indicate any major changes such as adding, deleting, or moving requirements. This also helped with the overall readability of the document.

2 Deliverable Changes

2.1 Requirements

2.1.1 Requirements Gathering & Analysis

The requirements gathering and analysis structure that *Team 3* implemented is very systematic and methodical, following a step-by-step process to ensure clarity. This approach proved effective for *Team 3* in Assessment 1 which was directly applied to similar tasks in Assessment 2. With this success, we decided to keep this approach for Assessment 2, as they provided a reliable framework and a strong single statement of need that continued to align with project goals. The only change considered was figuring out whether another stakeholder meeting was necessary, which we decided was not necessary because the product brief clearly outlined any additions.

2.1.2 User Requirements

When approaching the user requirements we reviewed the original and updated product brief to see whether any changes needed to be made to expand on new elements of the game. We came to the conclusion that no new user requirements needed to be added to compensate for new functionalities (though we require expansion in functional & non-functional requirements), then proceeded to review the usefulness of the user requirements and how they related to the functional and non-functional requirements. After our observations we came to the conclusion that many of the team's original user requirements were "stand-in" requirements, in other words, solely there for the purpose of having a user requirement to relate to a functional requirement created.

To mitigate this, we reviewed the "stand-in" requirements to see what they have in common, and they sorted themselves into three categories: UR_USABILITY, UR_UI, and UR_LIMIT.

1. **UR_USABILITY:** The game shall provide clear instructions and prompts to support the user throughout gameplay.
 - UR_PAUSE
 - UR_SETTINGS
 - UR_SHORT_CUTS
 - UR_TEACH_DURING_GAME
 - UR_QUICK_START
2. **UR_UI:** The game will provide an interface for the user to interact with the game.
 - UR_BASIC_UI
 - UR_FLEXIBLE_UI
 - UR_CLEAR_DISPLAY
 - UR_DEVICE_ACCESSIBILITY
3. **UR_LIMIT:** The game shall establish clear boundaries and limitations in the game, such as time constraints and map boundaries that would affect the overall gameplay experience for the user
 - UR_FIVE_MINUTES

By grouping the original stand-in requirements and assigning the more detailed user requirements to functional requirements, we follow best practices, having broad user needs that will then be supported by detailed functional requirements. Though the user requirements the original team created were valuable, we agreed that they would be valuable elsewhere, and not just be placeholders to justify functional requirements. This consolidation method will have the testing and implementation team down the line, preventing confusion by making a clear distinction between high-level user goals and the functional behaviors of the system.

2.1.3 Functional Requirements

The changes in functional requirements started with adding some user requirements such as FR_SHORT_CUTS, FR_SETTINGS, and FR_PAUSE, connecting them to the new

consolidated requirement UR_USABILITY. The next steps that we took were to go through the original functional requirements, looking for any redundancies, adding functionalities the original team missed, and fixing the user requirements column to reflect the new requirements made. After fixing the functional requirements, we reviewed the product brief once more and decided to add two more to help us continue into Assessment 2.

1. Consolidation of Redundant Requirements

- FR_OPERATING_SYSTEMS was removed because its purpose was already covered under a non-functional requirement that covers system performance based on operating systems.
- FR_TEXTUAL_DESCRIPTIONS was redundant as there are other requirements such as FR_HOVER_INFO and FR_TIMER that describe the necessary textual prompts that are being displayed.
- FR_MUTE was integrated into FR_ALTER_VOLUME by including the range of volume being from maximum to mute.

2. New Functionalities for Assessment 2

- FR_ACHIEVEMENTS: covers achievements given out during the game based on the user's interactions that can change the satisfaction score
- FR_LEADERBOARD: covers displaying a leaderboard of the top five scores

We expanded the original team's building and event functionalities to address gaps identified in the product brief. For the building functionalities, the original requirements covered placing and moving buildings and roads, but they lacked critical elements such as deleting buildings, building types, and defining map restrictions. To address this, FR_DELETE_ELEMENTS, FR_BUILDING_TYPES, and FR_BUILD_RESTRICTIONS were added to ensure the game's playability reflects what is outlined in the project brief. To note, NFR_VARIETY, which covers building types, was reclassified as a functional requirement (FR_BUILDING_TYPES) since it specifies system capabilities. For event functionalities, we expanded it to include event triggers by factors such as time or user actions, and these events can be positive, negative, or neutral. This behavior was formalised in FR_EVENT_TRIGGER to best reflect the original product brief.

2.1.4 Non-functional Requirements

This final section required minimal changes, only a few clarifying descriptions, merging similar requirements, and altering the "fit criteria" section to align with optimal responses during user evaluations. For example, NFR_FAST_LOAD and UR_QUICK_START, both non-functional requirements, both describe how game assets are loaded up before gameplay, were merged. Similarly, three other user requirements, UR_TEACH_DURING_GAME, UR_DEVICE_ACCESSIBILITY, and UR_CLEAR_DISPLAY, were reclassified as non-functional requirements for further clarity.

UR_TEACH_DURING_GAME was simplified to NFR_INTUITIVE to make the referencing more concise, emphasizing that the game's systems and controls should be cohesive and easy to understand. UR_DEVICE_ACCESSIBILITY and FR_OPERATING_SYSTEMS were merged into NFR_PLATFORM_COMPATIBILITY, which falls under UR_UI and describes system performance across devices and operating systems, making it non-functional. Lastly, UR_CLEAR_DISPLAY was combined with NFR_CLEAR_GRAPHICS, still describing the graphics quality of the game. To enhance the table, we added two more non-functional requirements, NFR_ERROR_HANDLING, ensuring the system can manage bugs effectively, and NFR_ACCESSIBLE, addressing accessibility issues for the user. These

additions strengthen the quality and usability of the system for Assessment 2, especially for Testing and User Evaluations.

2.2 Architecture

2.2.1 Justification for lack of major changes

When reviewing *Team 3's* Architecture for Assessment 1 we felt that it largely satisfied the brief's requirements. Section 6.2 of *Team 3's* Architecture gave multiple *"diagrammatic representations (structural and behavioural diagrams) of the architecture of the team's product"*, as seen in Figures 1-6. In this section they also gave *"a brief statement of the specific languages (for instance, relevant parts of UML) and the tool(s) used to create these representations."* There they stated that the Python Graphviz module was used as their initial tool to create the representations Figure 1 and 2.

Team 3 then stated that they used the UML tool Lucidchart in order to create the more in depth OOP class diagram Figure 3. As Figure 3 was an updated version of Figure 1, this demonstrates how *Team 3's* architecture *"was initially designed and how it evolved over the course of the project."*

Team 3's architecture also thoroughly meets the brief's requirement to *"include a systematic justification for this architecture"*. This is firstly evident in Section 1, which discusses the hybrid architectural style chosen, combining MVC and ECS. Then in Section 2 the core architecture is described; the Game Engine Layer, Map Module, Student Satisfaction Module, Event System, UI System and Input Management. Section 3 further explains the additional features in *Team 3's* architecture, like data persistence, modularity, scalability, and cross-platform compatibility. Finally, Section 5 discusses the tradeoffs made in their architecture.

Sections 2, 3 and 6 satisfy the requirement to *"Relate the architecture clearly to the requirements, using your requirements referencing for identification"*. *Team 3's* Functional and Non-Functional requirements are consistently referenced across these sections. Finally, *Team 3's* architecture is *"≤ 6 pages"*, fulfilling the brief's length requirement. Therefore we felt that it was ample in length and needed no major supplement.

2.2.2 Updated Architecture Section on Website

The only major requirement that *Team 3's* architecture did not satisfy however was not *"provid[ing] evidence of the design process followed (e.g. interim versions of architectural diagrams, CRC cards) on your team's website and link to them from your report"*. Therefore, to demonstrate this design process, Figure 1 from Section 6.2 was added to the 'Architecture Extra' section of *Team 3's* website, along with a brief explanation of the interim diagram and how it evolved.

2.2.3 Additional Assessment 2 Section

To accommodate for the additional features *Team 1* would add onto *Team 3's* existing Assessment 1 architecture, we added a supplementary Assessment 2 section (page 8 onwards). Instead of using *Team 3's* preferred UML tools (Python GraphWiz and Lucidchart), we decided to use PlantUML class diagrams, as we are more familiar with the use of this UML tool from Assessment 1.

The **added class diagrams** are as follows:

- An updated version of *Team 2's* Figure 3 diagram, with an additional recreational building

- Scenario Class (Events) Diagrams
- An Achievements, Events and Score Class Diagram
- A new Menu Screen Classes Diagram
- A new UI Classes, for Achievements and Satisfaction Diagram

2.3 Method Selection & Planning

2.3.1 Software Engineering Methods

For Assessment 2, we chose to keep the Agile Scrum framework as our software engineering method due to its effectiveness in promoting collaboration, adaptability, and continuous improvement. The iterative nature of Scrum, dividing the task deliverables into manageable sprints helped with the multiple phases laid out within the project. This approach gave us the ability to adapt to new challenges that arose in Assessment 2, especially with regular sprint meetings to track progress, and highlight possible delays, recalibrating when necessary. Furthermore, the success of *Team 3* in Assessment 1 shows that this method will manage the growing demands of Assessment 2. However, despite keeping the method, we changed a few of the development and collaboration tools to better reflect our team's dynamic.

The primary communication platform was changed from Discord to Whatsapp Communities, the platform we used in Assessment 1. The decision was driven by its accessibility and ease of use, allowing team members to respond quickly, reducing any possibility for missed communications. The Communities feature was also a major factor, supporting the creation of separate communication channels categorised by deliverables and a general chat for overall group communication. This layout minimised the amount of messages missed and made it easier for group members to review previous deliverable-specific discussions if they are added to it later on.

Project management tools transitioned from PlantUML diagrams to Jira because we wanted a strong foundational platform that provided more in-depth workflow reporting. It also offered features such as issue tracking, activity logs, dashboards, with clear visibility of deadlines and dependencies within each sprint. From this platform we were able to assign tasks more effectively, overall enhancing our ability to keep the team on track and organised.

2.3.2 Team Organisation

In Assessment 2, our team adopted a more methodical approach to organization, focusing on the practical elements in comparison to theoretical frameworks. Building upon insights gained during Assessment 1, we prioritised role allocation based on demonstrated strengths and areas of growth, supported by our cross-functional learning approach we used prior. *Team 3* used Myers-Briggs and York Strengths to create a foundational understanding of potential team dynamics before discussing the specific skills and capabilities required to complete each of the project deliverables. While this is a valuable approach for a team who are unfamiliar with each other, we had the advantage in Assessment 2 of familiarity and prior performance knowledge. This overall helped us to make more informed decisions from week one of the project, as assigning roles and responsibilities came easily knowing everyone's demonstrated skills and preferences.

We agreed with *Team 3*'s strategy to allocate at least two people for each deliverable section to ensure a low bus factor and mitigate risk, as well as adding a team lead to each deliverable. Having a lead for each deliverable was beneficial because it provided a clear point of accountability and oversight, someone who had a comprehensive understanding of all the deliverable aspects. This approach was valuable when we had instances of team members unable to begin work on their deliverable at the same time as others, so the

deliverable lead could bring them up to speed. In Assessment 1 we recognised this as an area of weakness that we did not wish to repeat in Assessment 2, instead paving the way for more effective communication and coordination to track progress and solve issues before they led to risk. We also appointed a project manager, to oversee meetings and overall project progress to ensure we were reaching objectives.

2.3.3 Team Plan

The original systematic plan created by *Team 3* for Assessment 1 was designed to fit the specific deliverables outlined in that section of the assessment document. However, since Assessment 2 introduces a new set of deliverables with differing complexity and scope, it is necessary to create a revised plan, an addendum to the original plan. Using the exact plan from Assessment 1 would not be sufficient as it only addresses the requirements of that assessment, so following it would lead to skewed outcomes.

Instead we chose to create a continuation of the phased structure followed by the original plan. Assessment 1 incorporates phases one through six, and we extended the plan by adding phases seven through thirteen for Assessment 2. This approach overall helped with continuity by maintaining the same format within the plan document, so we could visualise how the plan developed across assessments.

2.4 Risk Assessment & Mitigation

2.4.1 Risk Management Process

For Assessment 2, we decided to keep the same risk management process, and Risk Register format. We felt that *Team 3*'s management process was described in good detail, matching *Team 1*'s Assessment 1 four-stage process of Risk Identification, Risk Analysis, Risk Planning and subsequent Risk Monitoring.

Additionally, the Risk Register format was decided to remain consistent with *Team 3*'s Assessment 1 format. The seven column format of Risk ID, Risk Type, Risk Description, Risk Likelihood, Risk Severity, Risk Mitigation and Risk Owner was identical to our team's Assessment 1 Risk Register format. This, therefore, made it straightforward to read, edit, and build upon for Assessment 2.

2.4.2 Edits to Assessment 1 Risk Register

When reviewing *Team 3*'s Assessment 1 Risk Register however, we felt that substantial improvement was needed. Upon reviewing the Risk Types column, we identified several instances where risk types were incorrectly matched to their respective risk descriptions. The following changes were made to correct these errors:

- R3: Updated from "Project" to "Product and Project."
- R5: Updated from "Product" to "Business."
- R7: Updated from "Business" to "Project."

Additionally, we felt that several of *Team 3*'s Risk Description and Risk Mitigation columns were lacking in detail. Therefore, those risks which were similar to our Assessment 1 risks were integrated into the edited register. The following integrations were made:

- Our team's R7 was incorporated into *Team 1*'s R2.
- Our team's R11 was incorporated into *Team 1*'s R4.

Finally, we identified that *Team 3's* Risk Register only had 10 risks, only taking their Risk Register to < 2 pages. We felt that this did not satisfy the brief's requirement of ≤ 3 pages. To add more depth, we therefore appended the following risks from *Team 1's* risk register to the Assessment 1 Register:

- R11, *Team 1's* R8
- R12, *Team 1's* R3
- R13, *Team 1's* R4

2.4.2 Additional Assessment 2 Risk Register

In order to acknowledge the risks associated with working on *Team 3's* Assessment 1 project, we added an additional Risk Register section for Assessment 2. This section detailed potential risks associated with both the continuation of the project and any challenges that arose during its testing and integration. This new section increased the total length of the Risk Register to just under three pages, meeting the specified page limit.